

StabCert: translation validation for stabilizer channels

Frédéric Kramarczyk
Independent researcher
fkra62@gmail.com

Abstract

Quantum routing compilers rewrite circuits to satisfy hardware connectivity constraints, but the resulting circuits are seldom verified against the channel they are meant to implement. We present a decision procedure for the equality of stabilizer channels — Clifford unitaries with stabilizer ancilla preparation and partial trace — restricted to a specified input subspace.

Our procedure compares the canonical signed signatures of code-Choi states, relative to a support determined by the specification, and is both sound and complete: two such channels agree on the code subspace if and only if their signatures coincide. It runs in polynomial time, avoids dense matrix representations and basis state enumeration, and its verdict is invariant under any change of Stinespring dilation.

We implement this in StabCert, a tool that treats the compiler as an untrusted black box and certifies routed circuits produced by Qiskit SABRE and pytket against targets reconstructed from a specification rather than from a reference circuit. On the families studied, our measurements identify generator density — not circuit width alone — as a key factor in verification cost.

1 Introduction

Quantum programs written for hardware must be routed: a compiler inserts SWAP gates and relabels qubits so that every two-qubit operation acts on an edge of the device coupling map. This transformation is substantial, and it is performed by heuristic search procedures that are not themselves verified. Qiskit’s SABRE and pytket’s routing passes are widely deployed and largely trusted on the strength of testing.

Existing verification approaches address adjacent problems.

Compiler certification — proving the transformation itself, once and for all — applies only where the compiler’s source is available and under the verifier’s control. SABRE and pytket are neither.

Equivalence checking, as implemented for example in QCEC [?], compares a compiled circuit against a reference circuit. Like our approach it relies on canonical representations; the distinction is not canonicity but the object represented and the input required. Section 8 states it precisely, together with the capabilities and limits of the engines involved.

Deductive verification proves properties of programs written in a dedicated language, reaching algorithms as complex as Shor’s order finding, but takes its input as a program rather than as an artifact produced elsewhere.

None of these answers the question a routing pass raises in practice: given a circuit produced by an untrusted compiler, does it implement the channel the specification calls for? The distinction matters because the reference circuit is itself an artifact. Comparing against it certifies faithfulness to a possibly wrong starting point.

Contribution. We restrict attention to stabilizer channels, as defined in Section 2, and show that within this class the question is decidable in polynomial time. Specifically: